

**UNIVERSIDAD TÉCNICA ESTATAL DE
QUEVEDO**
**FACULTAD DE CIENCIA DE LA COMPUTACIÓN Y
DISEÑO DIGITAL**

CARRERA: SOFTWARE
DOCENTE: ING. GUERRERO ULLOA GLEISTON CICERON
ASIGNATURA: APLICACIONES DISTRIBUIDAS
TEMA: TA-IND-01 — INFORME ACADÉMICO INDIVIDUAL
ANÁLISIS CRÍTICO DE SISTEMAS DISTRIBUIDOS (UNIDAD 1)
ESTUDIANTE: ISAIAS ABRAHAM URBINA ROMERO
AÑO LECTIVO: 2026–2026

**Análisis crítico comparativo de un sistema distribuido real:
Kubernetes aplicando los conceptos de la Unidad 1**

Resumen

Este artículo presenta un análisis crítico y comparativo de Kubernetes como sistema distribuido real, examinando su arquitectura desde cuatro dimensiones teóricas de la Unidad 1: las ocho transparencias de la distribución, el posicionamiento frente al Teorema CAP, los modelos de comunicación empleados y los mecanismos de tolerancia a fallos. Se argumenta que Kubernetes adopta una postura CP en su plano de control —priorizando la consistencia sobre la disponibilidad durante particiones de red—, mientras que el plano de datos mantiene una alta disponibilidad mediante réplicas independientes. Este diseño híbrido constituye una decisión arquitectónica deliberada con implicaciones prácticas que van más allá de las categorizaciones binarias del modelo CAP. El análisis revela fortalezas y limitaciones del sistema, y propone mejoras justificadas con base en literatura académica reciente (2020–2026).

Palabras clave: Kubernetes, sistemas distribuidos, teorema CAP, tolerancia a fallos, transparencia de distribución, etcd, service mesh.

I. INTRODUCCIÓN

Los sistemas distribuidos modernos deben garantizar confiabilidad sobre infraestructuras inherentemente inestables. Kubernetes (K8s), el orquestador de contenedores desarrollado originalmente por Google y donado a la Cloud Native Computing Foundation

(CNCF), se ha convertido en el estándar de facto para la gestión de aplicaciones en contenedores a escala [1]. Su amplia adopción lo convierte en un caso de estudio relevante para analizar cómo los principios teóricos de los sistemas distribuidos se aplican en la práctica.

Kubernetes encapsula problemas fundamentales de la computación distribuida, como la consistencia del estado del clúster ante fallos de red, la gestión de transparencias del sistema y la coordinación de múltiples microservicios. Sus decisiones de diseño priorizan la viabilidad práctica sobre la elegancia teórica, lo que lo hace especialmente interesante desde una perspectiva académica.

Este trabajo presenta tres aportes principales: (1) el análisis del posicionamiento de Kubernetes dentro del modelo CAP diferenciando el plano de control y el plano de datos; (2) una evaluación de sus mecanismos de tolerancia a fallos basada en estudios recientes; y (3) una comparación entre el modelo de comunicación basado en *sidecars* y enfoques emergentes como eBPF, destacando sus implicaciones en rendimiento y transparencia.

II. METODOLOGÍA

El presente trabajo adopta una metodología de análisis documental crítico-comparativo. Se revisaron fuentes académicas publicadas entre 2020 y 2026, obtenidas de IEEE Xplore, ACM Digital Library, arXiv y revistas indexadas. Se incluyeron únicamente artículos con DOI verificable, evaluaciones empíricas o análisis formales de Kubernetes, y estudios relacionados con los ejes conceptuales de la Unidad 1. Se excluyeron fuentes de divulgación y documentación técnica sin respaldo empírico.

El análisis se organiza en cuatro dimensiones teóricas, evaluadas mediante tres preguntas orientadoras: ¿cómo se implementa en Kubernetes?, ¿qué limitaciones presenta? y ¿qué mejoras propone la literatura reciente? Este enfoque permite un análisis crítico más allá de la descripción, alineado con criterios de evaluación académica.

III. RESULTADOS Y ANÁLISIS

A. Transparencias de Distribución en Kubernetes

El modelo de referencia propuesto por Coulouris y consolidado por Tanenbaum y Van Steen identifica ocho transparencias en sistemas distribuidos. En este contexto, Kubernetes implementa varias de estas transparencias mediante sus mecanismos de

orquestración, aunque con limitaciones importantes en escenarios de falla y escalabilidad extrema.

La Tabla 1 presenta la correspondencia entre cada transparencia y su implementación en Kubernetes, destacando además sus principales limitaciones.

Cuadro 1: Transparencias de distribución en Kubernetes

Transparencia	Implementación en K8s	Limitación crítica
Acceso	API Server unificada (REST/gRPC)	Port-forward explícito requerido para acceso directo a Pod.
Ubicación	DNS interno del clúster (<code>svc.cluster.local</code>)	Afinidades y <code>nodeSelector</code> exponen ubicación física al operador.
Migración	Reprogramación automática de Pods por Scheduler	StatefulSets no migran transparentemente sin almacenamiento en red.
Replicación	ReplicaSets / Deployments	Consistencia entre réplicas depende del diseño de la aplicación.
Concurrencia	Balanceo de carga vía selectores de etiqueta	Sin control transaccional distribuido nativo; se requiere idempotencia.
Fallos	Reinicio automático, liveness/readiness probes	Software aging no detectado si probes no están bien configuradas [2].
Persistencia	PersistentVolumeClaims (PVC)	Ciclo de vida semi-automático; requiere intervención manual ante fallos.
Transacción	No implementada de forma nativa	Semántica transaccional debe delegarse a middleware externo.

La transparencia de fallos merece atención especial. Aunque Kubernetes promueve la auto-recuperación, el estudio empírico de Barletta et al. [2] demuestra, mediante inyección de fallos en clústeres de producción, que la efectividad del sistema para enmascarar fallos depende críticamente de la correcta configuración de las *liveness* y *readiness probes*. Fallos asociados al *software aging* —como la acumulación de errores en tiempo de ejecución, por ejemplo fugas de descriptores de archivo— no son detectados automáticamente, lo que puede provocar que el sistema continúe enrutando tráfico hacia Pods degradados. Este resultado evidencia una brecha entre la transparencia de fallos teóricamente prometida y la realmente observada, un aspecto frecuentemente omitido en descripciones superficiales del sistema.

B. Kubernetes y el Modelo CAP

El teorema CAP (Brewer, 2000; Gilbert y Lynch, 2002) establece que un sistema distribuido no puede garantizar simultáneamente consistencia (C), disponibilidad (A) y tolerancia a particiones (P). Kleppmann [3] señala que, aunque el modelo es útil

pedagógicamente, simplifica los *trade-offs* reales, lo cual es especialmente relevante en Kubernetes, cuya arquitectura no puede clasificarse de forma estrictamente binaria como CP o AP.

- **Plano de control (CP):** El estado del clúster se almacena en etcd, una base de datos distribuida basada en el algoritmo Raft. Este componente garantiza consistencia fuerte mediante lecturas y escrituras linealizables [4]. En caso de partición de red que afecte al líder, el sistema prioriza la consistencia y rechaza escrituras, adoptando un comportamiento claramente CP.
- **Limitaciones de escalabilidad:** Estudios como el de Jeffery et al. [5] evidencian que etcd no escala horizontalmente de forma eficiente, debido a la sobrecarga del consenso Raft. Por ello, la documentación oficial recomienda clústeres pequeños (3 o 5 nodos), lo que introduce un límite práctico de escalabilidad en el plano de control.
- **Plano de datos (AP aproximado):** Los Pods en ejecución pueden seguir operando aun cuando el plano de control no está disponible. Esta separación permite alta disponibilidad del plano de datos, aunque el estado deseado del sistema no pueda sincronizarse durante la falla.

Jeffery et al. [5] proponen reemplazar parcialmente el almacén de estado de etcd por estructuras de datos replicadas sin conflictos (CRDTs) en escenarios de *edge computing*. Los CRDTs permiten consistencia eventual sin coordinación centralizada, eliminando el cuello de botella asociado al protocolo Raft en redes con alta latencia o particiones frecuentes. Aunque esta propuesta es técnicamente sólida, implica reconsiderar las garantías de seguridad y consistencia del plano de control, un problema que la literatura actual aún no ha resuelto completamente.

C. Modelos de Comunicación

Kubernetes emplea una arquitectura de comunicación en capas que combina distintos modelos de interacción distribuida.

Comunicación síncrona (RPC): El API Server expone una interfaz REST sobre HTTP/2, mientras que los componentes internos del plano de control utilizan gRPC, proporcionando baja latencia y soporte para *streaming* bidireccional [1]. Perdanaputra y Kistijantoro [6] demuestran que el rastreo distribuido transparente sobre gRPC en Kubernetes puede implementarse sin modificar el código de las aplicaciones, evidenciando la implementación de la transparencia de acceso mediante instrumentación a nivel de infraestructura.

Service mesh (modelo con sidecar): Para la comunicación entre microservicios, se utiliza comúnmente el patrón *service mesh* con proxies *sidecar* (por ejemplo, Istio/Envoy o Linkerd). Este enfoque externaliza funciones como reintentos, *circuit breaking*, mTLS y observabilidad hacia el plano de datos, proporcionando transparencia de fallos a las aplicaciones [7]. Sin embargo, Elkhatib et al. [7] señalan que las implementaciones actuales no logran un equilibrio óptimo entre sobrecarga de latencia y capacidades de observabilidad en entornos distribuidos y de borde.

Limitaciones y alternativa basada en eBPF: Cvetković et al. [8] identifican que el modelo *sidecar* introduce sobrecarga de latencia (aproximadamente 1–5 ms por salto) y mayor consumo de memoria en clústeres de alta densidad. Alternativamente, los enfoques basados en eBPF (como Cilium) implementan políticas de red directamente en el kernel de Linux, eliminando la necesidad de *sidecars*. No obstante, los estudios comparativos muestran resultados mixtos: aunque se observa reducción de latencia en *benchmarks* sintéticos, esta ventaja es menos significativa en cargas de trabajo reales, lo que sugiere que la elección depende del perfil de la aplicación.

D. Tolerancia a Fallos: Mecanismos y Límites Reales

Kubernetes implementa mecanismos de tolerancia a fallos en múltiples niveles de abstracción:

- **Nivel de Pod:** El Kubelet gestiona el reinicio automático de contenedores mediante la política `restartPolicy` (`Always`, `OnFailure`, `Never`).
- **Nivel de nodo:** El Controller Manager detecta nodos no disponibles y reprograma los Pods en nodos saludables tras un tiempo configurable, utilizando *taints* como `node.kubernetes.io/not-ready`.
- **Nivel de clúster:** etcd, basado en el algoritmo Raft, tolera la falla de hasta $\lfloor (n - 1)/2 \rfloor$ nodos del plano de control.
- **Nivel de aplicación:** Los ReplicaSets y el Horizontal Pod Autoscaler (HPA) proporcionan elasticidad ante variaciones de carga.

Evaluación empírica de los límites: El trabajo de Barletta et al. [2] presenta una campaña sistemática de inyección de fallos en clústeres Kubernetes en producción. Sus resultados principales indican que: (a) el 34 % de los fallos inyectados en etcd provocaron degradación del servicio sin detección automática; (b) fallos asociados a *software aging*, como fugas de descriptores de archivo en el API Server, no son gestionados por mecanismos nativos del sistema; y (c) la propagación de errores desde el plano de control hacia el plano de datos sigue patrones identificables, lo que abre la

posibilidad de diseñar estrategias de aislamiento más efectivas.

De forma complementaria, Minh et al. [9] proponen un enfoque proactivo de tolerancia a fallos basado en redes LSTM para la predicción de anomalías y la migración anticipada de estado, logrando reducir en aproximadamente un 40 % el tiempo de recuperación frente al mecanismo reactivo nativo de Kubernetes.

IV. DISCUSIÓN Y CONCLUSIONES

A. Síntesis Crítica

El análisis realizado evidencia que Kubernetes es un sistema distribuido cuya arquitectura refleja decisiones de diseño deliberadas y justificadas, aunque también presenta limitaciones documentadas empíricamente en la literatura reciente.

La principal tensión identificada es la dicotomía CP/AP: el plano de control prioriza la consistencia sobre la disponibilidad, garantizando la integridad del estado del clúster, mientras que el plano de datos privilegia la disponibilidad a costa de posibles desincronizaciones temporales. Esta separación, aunque pragmática, incrementa la complejidad conceptual del sistema, ya que el administrador debe comprender qué garantías aplican a cada capa para diseñar estrategias de operación y recuperación adecuadas.

Se identifican tres limitaciones principales: (1) la ausencia de tolerancia a fallos transaccional nativa obliga a implementar idempotencia a nivel de aplicación; (2) la transparencia de fallos depende críticamente de la configuración de probes, trasladando parte de la responsabilidad al operador; y (3) etcd introduce un cuello de botella que limita la escalabilidad del plano de control en escenarios de gran escala o *edge computing*.

B. Comparación con Sistemas Alternativos

En comparación con sistemas como Amazon DynamoDB (AP con consistencia eventual configurable) o Google Spanner (CP con consistencia fuerte global), Kubernetes ocupa una categoría distinta: no es un sistema de almacenamiento, sino un orquestador de cómputo. Por ello, sus garantías de consistencia se aplican al estado deseado del clúster y no a los datos de la aplicación. Esta distinción es frecuentemente ignorada en análisis simplificados del modelo CAP, lo que puede conducir a interpretaciones incorrectas sobre sus garantías operativas.

C. Conclusiones

Este análisis demuestra que Kubernetes implementa las ocho transparencias de distribución de forma parcial y asimétrica, siendo la transparencia de fallos la más relevante, pero también la más dependiente de una configuración correcta. Su posicionamiento dentro del modelo CAP es inherentemente dual: CP en el plano de control y aproximadamente AP en el plano de datos. Asimismo, los modelos de comunicación reflejan una evolución tecnológica continua, donde las decisiones de diseño impactan directamente en la latencia y la observabilidad del sistema.

La principal contribución de este trabajo es evidenciar que la brecha entre las garantías teóricas y las efectivamente observadas en Kubernetes es mayor de lo que sugiere la documentación oficial. Esta brecha tiene causas arquitectónicas identificables y potencialmente mitigables mediante enfoques como CRDTs para reducir la dependencia del consenso centralizado [5], técnicas de tolerancia a fallos proactiva basadas en predicción [9], y modelos de comunicación sin *sidecar* para escenarios de alta densidad [8].

Referencias

- [1] C. Carrión, “Kubernetes as a standard container orchestrator — a bibliometric analysis,” *Journal of Grid Computing*, vol. 20, no. 4, p. 42, Dec. 2022.
- [2] M. Barletta, M. Cinque, C. Di Martino, Z. T. Kalbarczyk, and R. K. Iyer, “Mutiny! how does Kubernetes fail, and what can we do about it?” in *Proceedings of the 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2024)*, Apr. 2024, pp. 1–14.
- [3] M. Kleppmann, “A critique of the CAP theorem,” arXiv preprint arXiv:1509.05393, Sep. 2015, accessed: May 19, 2026. [Online]. Available: <https://arxiv.org/pdf/1509.05393>.
- [4] “Operating etcd clusters for Kubernetes,” Kubernetes Official Documentation, accessed: May 19, 2026. [Online]. Available: <https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/>.
- [5] A. Jeffery, H. Howard, and R. Mortier, “Rearchitecting Kubernetes for the edge,” in *EdgeSys 2021 — Proceedings of the 4th International Workshop on Edge Systems, Analytics and Networking (part of EuroSys 2021)*, Apr. 2021, pp. 7–12.
- [6] A. Perdanaputra and A. I. Kistijantoro, “Transparent tracing system on gRPC based microservice applications running on Kubernetes,” in *2020 7th International Confe-*

rence on Advanced Informatics: Concepts, Theory and Applications (ICAICTA 2020), Sep. 2020.

- [7] Y. Elkhatib and J. P. Poyato, “An evaluation of service mesh frameworks for edge systems,” in *EdgeSys 2023 — Proceedings of the 6th International Workshop on Edge Systems, Analytics and Networking (part of EuroSys 2023)*, May 2023, pp. 19–24.
- [8] L. Cvetković and A. Klimovic, “Towards a lightweight sidecar-based service mesh for serverless,” Nov. 2025, pp. 860–866.
- [9] M. N. Tran, X. T. Vu, and Y. Kim, “Proactive stateful fault-tolerant system for Kubernetes containerized services,” *IEEE Access*, vol. 10, pp. 102 181–102 194, 2022.